УО Брестский Государственный Технический Университет
Кафедра ИИТ

Лабораторная работа №3
Вариант 19

Выполнил:
Студент 3 курса
Группы ПО-13
Тарасюк М.И.
Проверил:
Кулик А.Д.

Брест 2025

Цель: приобрести навыки применения паттернов проектирования при решении практических задач с использованием языка Python.

Задание 1:
Проект «Бургер-закусочная». Реализовать возможность формирования заказа из определенных позиций (тип бургера (веганский, куриный и т.д.)), напиток (холодный – пепси, кока-кола и т.д.; горячий – кофе, чай и т.д.), тип упаковки – с собой, на месте. Должна формироваться итоговая стоимость заказа.

```
class Order:
    def __init__(self):
        self.burgers = []
        self.drinks = []
        self.packaging = ""
        self.total_price = 0.0

    def add_burger(self, burger):
        self.burgers.append(burger)

    def add_drink(self, drink):
        self.drinks.append(drink)

    def set_packaging(self, packaging):
        self.packaging = packaging

    def add_to_price(self, price):
        self.total_price += price

    def __str__(self):
        result = "========== ВАШ ЗАКАЗ ==========\n"
        result += "Бургеры:\n"
        for burger in self.burgers:
            result += f"  • {burger}\n"
        result += "Напитки:\n"
        for drink in self.drinks:
            result += f"  • {drink}\n"
        result += f"Упаковка: {self.packaging}\n"
        result += f"ИТОГОВАЯ СТОИМОСТЬ: {self.total_price:.2f} руб.\n"
        result += "================================"
        return result


class PriceList:
    burger_prices = {}
    drink_prices = {}

    @classmethod
    def initialize(cls):
        cls.burger_prices = {
            "веганский": 6.50,
            "куриный": 7.00,
            "говяжий": 7.50,
            "рыбный": 7.20,
            "дабл бургер": 9.00,
            "чизбургер": 6.80,
            "детский": 5.50,
            "острый": 7.30
```

```
        }

        cls.drink_prices = {
            "кофе": 5.00,
            "капучино": 5.50,
            "латте": 5.50,
            "эспрессо": 4.50,
            "кола": 4.00,
            "кока-кола": 4.00,
            "пепси": 4.00,
            "фанта": 4.00,
            "спрайт": 4.00,
            "чай": 3.50,
            "зеленый чай": 3.50,
            "черный чай": 3.50,
            "морс": 4.50,
            "компот": 4.00,
            "вода": 2.00,
            "сок": 4.50
        }

    @classmethod
    def get_burger_price(cls, burger_type):
        return cls.burger_prices.get(burger_type.lower(), 7.00)

    @classmethod
    def get_drink_price(cls, drink_type):
        return cls.drink_prices.get(drink_type.lower(), 4.00)

    @classmethod
    def get_available_burgers(cls):
        return set(cls.burger_prices.keys())

    @classmethod
    def get_available_drinks(cls):
        return set(cls.drink_prices.keys())

    @classmethod
    def display_menu(cls):
        print("\n=== МЕНЮ БУРГЕР-ЗАКУСОЧНОЙ ===")
        print("БУРГЕРЫ:")
        for burger in sorted(cls.burger_prices.keys()):
            print(f"  • {burger:<12} - {cls.burger_prices[burger]:.2f} руб.")

        print("\nНАПИТКИ:")
        for drink in sorted(cls.drink_prices.keys()):
            print(f"  • {drink:<12} - {cls.drink_prices[drink]:.2f} руб.")

        print("\nУПАКОВКА: на месте / с собой")
        print("================================\n")


class OrderBuilder:
    def __init__(self):
        self.order = Order()
```

```
    def add_burger(self, burger_type):
        self.order.add_burger(burger_type)
        price = PriceList.get_burger_price(burger_type)
        self.order.add_to_price(price)
        return self

    def add_burgers(self, *burger_types):
        for burger in burger_types:
            self.add_burger(burger)
        return self

    def add_drink(self, drink_type):
        self.order.add_drink(drink_type)
        price = PriceList.get_drink_price(drink_type)
        self.order.add_to_price(price)
        return self

    def add_drinks(self, *drink_types):
        for drink in drink_types:
            self.add_drink(drink)
        return self

    def choose_packaging(self, packaging_type):
        self.order.set_packaging(packaging_type)
        return self

    def build(self):
        if not self.order.burgers:
            raise ValueError("Нужно добавить хотя бы один бургер!")
        if not self.order.drinks:
            raise ValueError("Нужно добавить хотя бы один напиток!")
        if not self.order.packaging:
            raise ValueError("Тип упаковки не выбран!")
        return self.order


def show_price_details(order):
    print("\n=== ДЕТАЛИЗАЦИЯ ЦЕН ===")
    total = 0

    print("Бургеры:")
    for burger in order.burgers:
        price = PriceList.get_burger_price(burger)
        print(f"  • {burger:<12}: {price:.2f} руб.")
        total += price

    print("Напитки:")
    for drink in order.drinks:
        price = PriceList.get_drink_price(drink)
        print(f"  • {drink:<12}: {price:.2f} руб.")
        total += price

    print(f"ИТОГО: {total:.2f} руб.")


def main():
```

```
# Инициализация цен
PriceList.initialize()

print("Добро пожаловать в Бургер-закусочную!\n")

PriceList.display_menu()

# Семейный заказ
family_order = OrderBuilder() \
    .add_burgers("говяжий", "куриный", "детский") \
    .add_drinks("кола", "фанта", "сок") \
    .choose_packaging("с собой") \
    .build()

print("ЗАКАЗ №1 (Семейный):")
print(family_order)
print()

# Заказ для компании
company_order = OrderBuilder() \
    .add_burger("дабл бургер") \
    .add_burger("чизбургер") \
    .add_burger("веганский") \
    .add_drinks("кофе", "чай", "пепси") \
    .choose_packaging("на месте") \
    .build()

print("ЗАКАЗ №2 (Компания):")
print(company_order)
print()

# Индивидуальный заказ
individual_order = OrderBuilder() \
    .add_burger("рыбный") \
    .add_burger("острый") \
    .add_drinks("капучино", "вода") \
    .choose_packaging("с собой") \
    .build()

print("ЗАКАЗ №3 (Индивидуальный):")
print(individual_order)
print()

# Экономичный заказ
economy_order = OrderBuilder() \
    .add_burger("детский") \
    .add_drink("чай") \
    .choose_packaging("на месте") \
    .build()

print("ЗАКАЗ №4 (Экономичный):")
print(economy_order)
print()

# Тест неизвестных продуктов
print("=== ТЕСТ НЕИЗВЕСТНОГО ПРОДУКТА ===")
```

```
    try:
        test_order = OrderBuilder() \
            .add_burger("гамбургер") \
            .add_drink("лимонад") \
            .choose_packaging("с собой") \
            .build()
        print(test_order)
    except ValueError as e:
        print(f"Ошибка: {e}")


if __name__ == "__main__":
    main()
```

Задание 2:
Проект «Часы». В проекте должен быть реализован класс, который дает возможность пользоваться часами со стрелками так же, как и цифровыми часами. В классе «Часы со стрелками» хранятся повороты стрелок.

```
from abc import ABC, abstractmethod

# Интерфейс цифровых часов
class DigitalClockInterface(ABC):
    @abstractmethod
    def get_time_string(self):
        pass

    @abstractmethod
    def set_time(self, hours, minutes, seconds):
        pass

# Класс стрелочных часов
class AnalogClock:
    def __init__(self):
        self.second_angle = 0
        self.minute_angle = 0
        self.hour_angle = 0

    def tick(self):
        """Проходит 1 секунда"""
        self.second_angle = (self.second_angle + 6) % 360

        if self.second_angle == 0:
            self.minute_angle = (self.minute_angle + 6) % 360

            if self.minute_angle == 0:
                self.hour_angle = (self.hour_angle + 30) % 360

    def set_angles(self, hour_angle, minute_angle, second_angle):
        self.hour_angle = hour_angle % 360
        self.minute_angle = minute_angle % 360
        self.second_angle = second_angle % 360

    def get_angles(self):
        return self.hour_angle, self.minute_angle, self.second_angle
```

```python
# Адаптер
class ClockAdapter(DigitalClockInterface):
    def __init__(self, analog_clock):
        self.clock = analog_clock

    def set_time(self, hours, minutes, seconds):
        """Преобразует время в углы"""
        self.clock.set_angles(
            hours * 30,
            minutes * 6,
            seconds * 6
        )

    def get_time_string(self):
        """Преобразует углы во время"""
        h_angle, m_angle, s_angle = self.clock.get_angles()

        hours = h_angle // 30
        minutes = m_angle // 6
        seconds = s_angle // 6

        return f"{hours:02d}:{minutes:02d}:{seconds:02d}"

    def tick(self):
        self.clock.tick()

def main():
    print("ЧАСЫ")
    print("-" * 30)

    # Создаем часы
    analog_clock = AnalogClock()
    digital_clock = ClockAdapter(analog_clock)

    # Устанавливаем время
    print("Установка времени 10:30:45")
    digital_clock.set_time(10, 30, 45)

    # Проверяем
    print(f"Цифровое время: {digital_clock.get_time_string()}")
    h, m, s = analog_clock.get_angles()
    print(f"Углы стрелок: час {h}°, мин {m}°, сек {s}°")

    # Проходит 15 секунд
    print("\nПроходит 15 секунд...")
    for _ in range(15):
        digital_clock.tick()

    print(f"Текущее время: {digital_clock.get_time_string()}")
    h, m, s = analog_clock.get_angles()
    print(f"Углы стрелок: час {h}°, мин {m}°, сек {s}°")

if __name__ == "__main__":
    main()
```

Задание 3:
Шифрование текстового файла. Реализовать класс-шифровщик текстового файла с поддержкой различных алгоритмов шифрования. Возможные варианты шифрования: удаление всех гласных букв из текста, изменение букв текста на буквы, получаемые фиксированным сдвигом из алфавита (например, шифром буквы а будет являться буква д для сдвига 4 и т.д.), применение операции исключающее или с заданным ключом.

```
from abc import ABC, abstractmethod
import os
from typing import Optional

class EncryptionStrategy(ABC):
    """Абстрактный класс стратегии шифрования"""

    @abstractmethod
    def encrypt(self, text: str) -> str:
        """Зашифровать текст"""
        pass

    @abstractmethod
    def decrypt(self, text: str) -> str:
        """Расшифровать текст"""
        pass

    @property
    @abstractmethod
    def name(self) -> str:
        """Название алгоритма"""
        pass


# ============= Конкретные стратегии =============
class VowelRemovalStrategy(EncryptionStrategy):
    """Стратегия: удаление всех гласных букв"""

    def __init__(self):
        self._vowels = set('aeiouyaeёиоуыэюяAEIOUYAEЁИОУЫЭЮЯ')

    def encrypt(self, text: str) -> str:
        return ''.join(char for char in text if char not in self._vowels)

    def decrypt(self, text: str) -> str:
        return f"[Невозможно восстановить гласные] {text}"

    @property
    def name(self) -> str:
        return "Удаление гласных"


class CaesarShiftStrategy(EncryptionStrategy):
    """Стратегия: сдвиг букв на фиксированное число"""

    def __init__(self, shift: int = 20):
        self.shift = shift
```

```
        self.ru_lower = 'абвгдеёжзийклмнопрстуфхцчшщъыьэюя'
        self.ru_upper = 'АБВГДЕЁЖЗИЙКЛМНОПРСТУФХЦЧШЩЪЫЬЭЮЯ'

        self.en_lower = 'abcdefghijklmnopqrstuvwxyz'
        self.en_upper = 'ABCDEFGHIJKLMNOPQRSTUVWXYZ'

    def _shift_char(self, char: str, alphabet: str, shift: int) -> str:
        """Сдвигает символ в пределах алфавита"""
        if char in alphabet:
            idx = alphabet.index(char)
            new_idx = (idx + shift) % len(alphabet)
            return alphabet[new_idx]
        return char

    def encrypt(self, text: str) -> str:
        """Сдвигает буквы вперед на shift позиций"""
        result = []
        for char in text:
            if char in self.ru_lower:
                result.append(self._shift_char(char, self.ru_lower, self.shift))
            elif char in self.ru_upper:
                result.append(self._shift_char(char, self.ru_upper, self.shift))
            elif char in self.en_lower:
                result.append(self._shift_char(char, self.en_lower, self.shift))
            elif char in self.en_upper:
                result.append(self._shift_char(char, self.en_upper, self.shift))
            else:
                result.append(char)
        return ''.join(result)

    def decrypt(self, text: str) -> str:
        """Сдвигает буквы назад на shift позиций"""
        result = []
        for char in text:
            if char in self.ru_lower:
                result.append(self._shift_char(char, self.ru_lower, -self.shift))
            elif char in self.ru_upper:
                result.append(self._shift_char(char, self.ru_upper, -self.shift))
            elif char in self.en_lower:
                result.append(self._shift_char(char, self.en_lower, -self.shift))
            elif char in self.en_upper:
                result.append(self._shift_char(char, self.en_upper, -self.shift))
            else:
                result.append(char)
        return ''.join(result)

    @property
    def name(self) -> str:
        return f"Сдвиг на {self.shift}"


class XorStrategy(EncryptionStrategy):
    """Стратегия: XOR с заданным ключом"""

    def __init__(self, key: str = "чебурашка"):
        self.key = key
```

```
        # Преобразуем ключ в байт
        self.key_bytes = key.encode('utf-8')

    def _xor_operation(self, data: bytes) -> bytes:
        key_len = len(self.key_bytes)
        result = bytearray()

        for i, byte in enumerate(data):
            result.append(byte ^ self.key_bytes[i % key_len])

        return bytes(result)

    def encrypt(self, text: str) -> str:
        """
        Шифрует текст с помощью XOR
        Возвращает строку в шестнадцатеричном представлении
        """
        text_bytes = text.encode('utf-8')
        encrypted_bytes = self._xor_operation(text_bytes)
        return encrypted_bytes.hex()

    def decrypt(self, text: str) -> str:
        """
        Расшифровывает текст из шестнадцатеричного представления
        """
        try:
            encrypted_bytes = bytes.fromhex(text)
            decrypted_bytes = self._xor_operation(encrypted_bytes)
            return decrypted_bytes.decode('utf-8')
        except (ValueError, UnicodeDecodeError) as e:
            return f"[Ошибка расшифровки: {e}]"

    @property
    def name(self) -> str:
        return f"XOR (ключ: {self.key})"


# ============= Контекст (класс-шифровщик) =============
class FileEncryptor:
    """Шифровщик текстовых файлов"""

    def __init__(self, strategy: Optional[EncryptionStrategy] = None):
        """
        :param strategy: стратегия шифрования
        """
        self._strategy = strategy

    def set_strategy(self, strategy: EncryptionStrategy) -> None:
        """Сменить стратегию шифрования"""
        self._strategy = strategy

    def encrypt_file(self, input_path: str, output_path: str) -> bool:
        """
        Зашифровать файл

        :param input_path: путь к исходному файлу
```

```
    :param output_path: путь к зашифрованному файлу
    :return: True в случае успеха, False при ошибке
    """
    if not self._strategy:
        raise ValueError("Стратегия шифрования не установлена")

    try:
        with open(input_path, 'r', encoding='utf-8') as f:
            content = f.read()

        encrypted_content = self._strategy.encrypt(content)

        with open(output_path, 'w', encoding='utf-8') as f:
            f.write(encrypted_content)

        return True

    except FileNotFoundError:
        print(f"Ошибка: файл {input_path} не найден")
        return False
    except Exception as e:
        print(f"Ошибка при шифровании: {e}")
        return False

def decrypt_file(self, input_path: str, output_path: str) -> bool:
    """
    Расшифровать файл

    :param input_path: путь к зашифрованному файлу
    :param output_path: путь к расшифрованному файлу
    :return: True в случае успеха, False при ошибке
    """
    if not self._strategy:
        raise ValueError("Стратегия шифрования не установлена")

    try:
        with open(input_path, 'r', encoding='utf-8') as f:
            content = f.read()

        decrypted_content = self._strategy.decrypt(content)
        with open(output_path, 'w', encoding='utf-8') as f:
            f.write(decrypted_content)

        return True

    except FileNotFoundError:
        print(f"Ошибка: файл {input_path} не найден")
        return False
    except Exception as e:
        print(f"Ошибка при расшифровке: {e}")
        return False

def get_current_strategy_name(self) -> str:
    """Получить название текущей стратегии"""
    return self._strategy.name if self._strategy else "Не установлена"
```

```python
def main():
    """Демонстрация работы шифровщика"""

    # Создаем тестовый файл
    test_file = "test_input.txt"
    with open(test_file, 'w', encoding='utf-8') as f:
        f.write("Привет, мир! Hello, world!\n")
        f.write("Это тестовый файл для шифрования.\n")
        f.write("XOR encryption test with key: чебурашка")

    encryptor = FileEncryptor()

    # ===== Тест 1: Удаление гласных =====
    print("=" * 50)
    print("Тест 1: Удаление гласных")
    encryptor.set_strategy(VowelRemovalStrategy())
    print(f"Стратегия: {encryptor.get_current_strategy_name()}")

    encryptor.encrypt_file("test_input.txt", "test_vowel_removed.txt")
    encryptor.decrypt_file("test_vowel_removed.txt", "test_vowel_decoded.txt")

    with open("test_vowel_removed.txt", 'r', encoding='utf-8') as f:
        print("Зашифровано:")
        print(f.read())

    # ===== Тест 2: Сдвиг на M =====
    print("\n" + "=" * 50)
    print("Тест 2: Сдвиг на 3")
    encryptor.set_strategy(CaesarShiftStrategy(shift=3))
    print(f"Стратегия: {encryptor.get_current_strategy_name()}")

    encryptor.encrypt_file("test_input.txt", "test_caesar.txt")
    encryptor.decrypt_file("test_caesar.txt", "test_caesar_decoded.txt")

    print("Расшифрованный файл:")
    with open("test_caesar_decoded.txt", 'r', encoding='utf-8') as f:
        print(f.read())

    # ===== Тест 3: XOR с ключом =====
    print("\n" + "=" * 50)
    print("Тест 3: XOR с ключом 'чебурашка'")
    encryptor.set_strategy(XorStrategy(key="чебурашка"))
    print(f"Стратегия: {encryptor.get_current_strategy_name()}")

    encryptor.encrypt_file("test_input.txt", "test_xor.txt")

    with open("test_xor.txt", 'r', encoding='utf-8') as f:
        print("Зашифровано (hex):")
        print(f.read()[:100] + "...")

    encryptor.decrypt_file("test_xor.txt", "test_xor_decoded.txt")

    print("\nРасшифрованный файл:")
    with open("test_xor_decoded.txt", 'r', encoding='utf-8') as f:
        print(f.read())
```

```
    # ===== Тест 4: Смена стратегии на лету =====
    print("\n" + "=" * 50)
    print("Тест 4: Смена стратегии на лету")
    encryptor.set_strategy(CaesarShiftStrategy(shift=10))
    print(f"Новая стратегия: {encryptor.get_current_strategy_name()}")
    encryptor.encrypt_file("test_input.txt", "test_caesar_10.txt")
    print("Файл зашифрован со сдвигом 10")

    encryptor.decrypt_file("test_caesar_10.txt", "test_caesar_10_decoded.txt")

    print("Расшифрованный файл:")
    with open("test_caesar_10_decoded.txt", 'r', encoding='utf-8') as f:
        print(f.read())

if __name__ == "__main__":
    main()
```

Вывод: приобретены навыки применения паттернов проектирования при решении практических задач с использованием языка Python.